forave

ESCOLA PROFISSIONAL TECNOLÓGICA
DO VALE DO AVE

ANALISADOR DE LOGS DE SISTEMA

Relatório de Projeto Prático

Cláudio Tolentino Xavier Machado

UFCD 10806
Linguagem de Scripts e Linha de Comandos

Prof. Carlos Vicente

Abril 2026

Abril 2026

1. Enquadramento

No âmbito da UFCD 10806 — Linguagem de Scripts e Linha de Comandos, foi desenvolvido um projeto prático com o objetivo de simular o papel de um analista de sistemas. A tarefa consistiu em analisar um ficheiro de logs gerado por um sistema informático, utilizando exclusivamente ferramentas da linha de comandos (CLI) e scripting em Bash.

O ficheiro de logs fornecido — 10000.log — contém 10 000 registos de atividade, incluindo ações de login, logout, download, upload, acesso e erros. Cada linha segue a estrutura:

```
<timestamp>  <IP>  <utilizador>  <ação>  <recurso>  <status>
Exemplo: 2026-03-01T08:00:00 192.168.1.12 user3 LOGIN /login OK
```

2. Objetivos

O projeto foi desenvolvido ao longo de várias aulas, com objetivos progressivos:

- Aula 2 — Análise manual com comandos CLI (grep, awk, sort, uniq, wc)
- Aula 3 — Criação de um script simples de relatório (analyzer.sh)
- Aula 4 — Evolução do script: validação, loop, menu interativo e geração de ficheiro
- Aula 5 — Refatoração com funções, filtros por utilizador/ação e automação

3. Estrutura do Projeto

O repositório Git do projeto está organizado da seguinte forma:

```
projeto/
├── analyzer.sh          # Versão 1 – relatório simples (Aula 3)
├── analyzer2.sh         # Versão 2 – contagem de erros com loop (Aula 4)
├── analyzer3.sh         # Versão 3 – menu interativo (Aula 4)
├── analyzer_final.sh    # Versão final – completa (Aula 5)
├── auto_analyzer.sh     # Script de automação (Aula 5)
├── 10000.log            # Ficheiro de logs para análise
├── relatorio_2026-04-25.txt  # Relatório gerado automaticamente
└── README.md            # Documentação do projeto
```

4. Desenvolvimento por Aulas

4.1 Aula 2 — Análise com Comandos CLI

Nesta fase foram utilizados comandos da linha de comandos para responder às questões de análise do ficheiro de logs. Os principais comandos utilizados foram:

Questão	Comando
Total de linhas	wc -l 10000.log
Total de erros	grep -c ' ERROR ' 10000.log
IPs únicos	awk '{print $2}' 10000.log | sort | uniq | wc -l
Utilizadores únicos	awk '{print $3}' 10000.log | sort | uniq | wc -l
Top IP	awk '{print $2}' | sort | uniq -c | sort -nr | head -1
Recurso mais acedido	awk '{print $5}' | sort | uniq -c | sort -nr | head -1
Downloads	grep -c ' DOWNLOAD ' 10000.log
Uploads	grep -c ' UPLOAD ' 10000.log
Erros 500	grep -c ' 500$' 10000.log

4.2 Aula 3 — Primeiro Script (analyzer.sh)

O primeiro script foi criado para gerar automaticamente um relatório simples a partir do ficheiro de logs. Apresenta no terminal as estatísticas essenciais e guarda o resultado num ficheiro de texto. Inclui também validação básica do argumento de entrada.

4.3 Aula 4 — Script Evoluído (analyzer2.sh e analyzer3.sh)

Na aula 4 foram introduzidas três melhorias ao script original:

- Desafio 1 — Validação: verifica se foi passado um argumento e se o ficheiro existe, terminando com mensagem de erro caso contrário.
- Desafio 2 — Loop: contagem de erros usando while read line com condição if, em vez de grep -c.
- Desafios 3 e 4 — Menu interativo: apresenta um menu ao utilizador com 7 opções, repetindo-se em loop até o utilizador escolher sair.
- Desafio 5 — Relatório em ficheiro: opção no menu que gera o ficheiro relatorio.txt com todas as estatísticas.

4.4 Aula 5 — Script Final (analyzer_final.sh)

O script final representa a versão mais completa e profissional do projeto, organizado em 6 secções bem definidas:

- Secção 1 — Validação de argumentos e aplicação de filtros opcionais (user= e action=)
- Secção 2 — Funções reutilizáveis para cada funcionalidade de análise
- Secção 3 — Geração de relatório completo em ficheiro com data no nome
- Secção 4 — Menu interativo com 16 opções e cores no terminal
- Secção 5 — Modo automático (--auto) para execução sem interação
- Secção 6 — Loop principal do menu

O auto_analyzer.sh complementa o script final, permitindo executar a análise de forma completamente automática, ideal para agendamento ou integração em pipelines.

5. Resultados da Análise

Após a execução do analyzer_final.sh sobre o ficheiro 10000.log, obtiveram-se os seguintes resultados:

5.1 Estatísticas Gerais

Métrica	Valor
Total de linhas	10 000
Total de erros (ERROR)	1 662
IPs únicos	58
Utilizadores únicos	24

5.2 Atividade do Sistema

Tipo de Ação	Total
Downloads	1 726
Uploads	1 618
Erros com código 500	396

5.3 Rankings

Indicador	Resultado
IP com mais acessos	192.168.1.10 (208 acessos)
Utilizador mais ativo	user10 (442 ações)
Recurso mais acedido	/files/doc.pdf (1 709 vezes)
Utilizador com mais downloads	user14 (90 downloads)
IP com mais erros 500	192.168.1.40 (15 erros 500)
Recurso com mais erros	/api/data (286 erros)

5.4 Top 5 IPs com Mais Acessos

Posição	IP — Acessos
1.º	192.168.1.10 — 208 acessos
2.º	192.168.1.14 — 206 acessos
3.º	192.168.1.9 — 204 acessos
4.º	192.168.1.53 — 195 acessos
5.º	192.168.1.5 — 192 acessos

6. Funcionalidades Implementadas

O script final (analyzer_final.sh) implementa todas as funcionalidades pedidas, organizadas em funções independentes:

Função	Descrição
total_linhas()	Conta o total de linhas do ficheiro
total_erros()	Conta erros com while loop e if
ips_unicos()	Conta endereços IP únicos
total_downloads()	Conta eventos de download
total_uploads()	Conta eventos de upload
erros_500()	Conta erros com código 500
top_ip()	IP com mais acessos
top5_ips()	Top 5 IPs com mais acessos
utilizador_mais_ativo()	Utilizador com mais ações
recurso_mais_acedido()	Recurso mais acedido
user_mais_downloads()	Utilizador com mais downloads
ip_mais_erros500()	IP com mais erros 500
recurso_mais_erros()	Recurso que gera mais erros
gerar_relatorio()	Cria relatório em ficheiro com data

7. Conclusão

O desenvolvimento deste projeto permitiu consolidar competências essenciais na área da linha de comandos e scripting em Bash. Ao longo das aulas, o script evoluiu progressivamente de uma solução simples para uma ferramenta completa, modular e reutilizável.

As principais aprendizagens adquiridas foram:

- Utilização de comandos CLI como grep, awk, sort, uniq e wc para análise de dados.
- Criação de scripts Bash com validação de argumentos e tratamento de erros.
- Organização do código em funções reutilizáveis, tornando-o mais legível e manutenível.
- Implementação de menus interativos com case e loops while.
- Aplicação de filtros dinâmicos para análise segmentada por utilizador ou ação.
- Utilização do Git para controlo de versões e entrega do projeto.

O projeto atingiu todos os objetivos propostos pelo professor e vai além dos requisitos mínimos, incluindo funcionalidades adicionais como filtros combinados, modo automático e relatório com data no nome do ficheiro.